

BLBW

Be Life, Be Water

MANUAL TÉCNICO AVANÇADO DE INSTRUÇÕES E LÓGICA DE CONTROLE

Referência Rápida e Aplicações Práticas - Siemens TIA Portal / Step 7 (LAD/STL)

Versão: Corporativa 1.1

Público-Alvo: Engenharia de Automação e Manutenção Industrial (Nível Especialista)

Plataforma: CLP Siemens (S7-1200, S7-1500, S7-300/400)

Data: 03/07/2026

ÍNDICE GERAL

CAPÍTULO 1: Bit Logic Operations

- 1.1 Contatos de Alta Performance e Bobinas de Atribuição
- 1.2 Memorização de Estado e Flip-Flops (SR / RS)
- 1.3 Operações de Detecção de Borda (P_TRIG / N_TRIG / R_TRIG / F_TRIG)

CAPÍTULO 2: Timer Operations (IEC Timers)

- 2.1 TON (Timer On-Delay)
- 2.2 TOF (Timer Off-Delay)
- 2.3 TP (Timer Pulse)
- 2.4 TONR (Time Accumulator)

CAPÍTULO 3: Counter Operations (IEC Counters)

- 3.1 CTU (Count Up)
- 3.2 CTD (Count Down)
- 3.3 CTUD (Count Up/Down)

CAPÍTULO 4: Comparator Operations

- 4.1 Operações Matemáticas Relacionais (==, <>, >=, <=, >, <)
- 4.2 Checagem de Limites (IN_Range / OUT_Range)

CAPÍTULO 5: Math Functions

- 5.1 Cálculos Aritméticos Básicos (ADD, SUB, MUL, DIV)
- 5.2 Seleção de Extremos e Limitação (MIN, MAX, LIMIT)
- 5.3 Funções Trigonométricas e Exponenciais Avançadas

CAPÍTULO 6: Move Operations

- 6.1 Transferência de Bloco Escalar (MOVE)
- 6.2 Manipulação em Lote e Serialização (MOVE_BLK, FILL_BLK, SWAP)

CAPÍTULO 7: Conversion Operations

- 7.1 Conversão Explícita de Cast (CONVERT)
- 7.2 Arredondamento e Truncamento (ROUND, CEIL, FLOOR, TRUNC)
- 7.3 Escalonamento de Sinais Analógicos (SCALE_X, NORM_X)

CAPÍTULO 8: Program Control Operations

- 8.1 Saltos e Roteamento Lógico (JMP, JMPN, LABEL, SWITCH)
- 8.2 Finalização de Bloco (RET)

CAPÍTULO 9: Word Logic Operations

- 9.1 Mascaramento a Nível de Palavra (AND, OR, XOR)
- 9.2 Manipulação Estrutural (ENCO, DECO, SEL, MUX, DEMUX)

CAPÍTULO 10: Shift and Rotate Operations

- 10.1 Deslocamento de Bits (SHR, SHL)
- 10.2 Rotação de Registradores (ROR, ROL)

CAPÍTULO 1 (continuação): Bit Logic Operations

- 1.4 Inversão de RLO, Bobinas Especiais e Contatos de Borda
- 1.5 Operações em Campo de Bits (SET_BF / RESET_BF)

CAPÍTULO 2 (continuação): Timer Operations

- 2.5 Instruções Coil de Temporizador (-(TON)-, -(RT)-, -(PT)-)
- 2.6 Controle de Runtime (RE_TRIGR, RUNTIME)

CAPÍTULO 4 (continuação): Comparator Operations

- 4.3 Comparadores Variant e Ponteiros (EQ_Type, IS_NULL, IS_ARRAY)
- 4.4 Validação de Integridade (|OK| / |NOT_OK|)

CAPÍTULO 5 (continuação): Math Functions

- 5.4 Operações Modulares e Incrementais (MOD, INC, DEC, NEG, ABS)
- 5.5 Bloco CALCULATE — Expressões Compostas

CAPÍTULO 6 (continuação): Move Operations

- 6.3 Operações Ininterruptas e Bit Packing (UMOVE_BLK, SCATTER, GATHER)
- 6.4 Variant e Limites de Array (VariantGet, LOWER_BOUND, UPPER_BOUND)

CAPÍTULO 8 (continuação): Program Control

- 8.3 Lista de Saltos (JMP_LIST) e Diagnóstico Local (GET_ERROR)
- 8.4 Controle de Execução (STP, ENDIS_PW)

APÊNDICE A: Arquitetura de Execução e Ciclo de Scan

- A.1 Hierarquia de OBs e Prioridade de Saídas
- A.2 Process Image vs Acesso Direto (%I / %Q / PEEK / POKE)

APÊNDICE B: Referência Rápida LAD ↔ STL

- B.1 Tabela de Equivalência de Instruções
- B.2 Convenções de Nomenclatura BLBW

CAPÍTULO 1: Bit Logic Operations

1.1 Contatos de Alta Performance e Bobinas de Atribuição

Contato Normalmente Aberto (NA) — ||

Nome e Objetivo: Elemento de entrada que conduz corrente lógica (RLO = 1) quando o operando associado está em estado TRUE. Base de toda lógica combinacional em LAD.

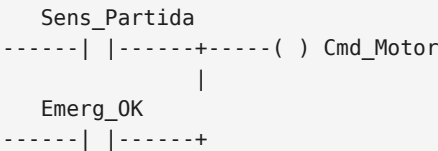
Quando usar: Leitura de sensores digitais (proximidade, pressostato, botoeira), habilitação de cadeias lógicas e condicionamento de blocos IEC. Em S7-1500, prefira tags otimizadas (%I, %Q) para acesso direto sem overhead de DB.

Sintaxe e Parâmetros: Operando (BOOL): Tag de entrada, memória M, bit de DB ou entrada física %I. O contato avalia o estado binário no início do ciclo de scan (process image input). Não possui pinos EN/ENO — é elemento de contato puro.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato NA AQUI]



STL (Statement List):

```
// Contato NA: instrução A (AND) / O (OR) conforme posição na rede
A    "Sens_Partida"
A    "Emerg_OK"
=    "Cmd_Motor"
```

Contato Normalmente Fechado (NF) — |/|

Nome e Objetivo: Elemento de entrada que conduz quando o operando está em FALSE. Inversão lógica de sinal de campo ou condição de segurança.

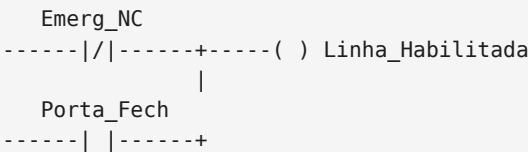
Quando usar: Intertravamentos de parada de emergência (NF em série), detecção de falta de sinal, lógica de alarme por perda de comunicação. Em STL, equivalente a AN (AND NOT).

Sintaxe e Parâmetros: Operando (BOOL): Mesmas regras do contato NA. A avaliação ocorre no ciclo atual; não há debounce nativo — implemente TON externo se necessário.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato NF AQUI]



STL (Statement List):

```
AN    "Emerg_NC"      // AND NOT – contato NF
A     "Porta_Fech"
=     "Linha_Habilitada"
```

Bobina de Atribuição — ()

Nome e Objetivo: Escreve o RLO atual no operando de saída a cada ciclo de scan. Comportamento volátil: se a condição de entrada cair, a saída retorna a FALSE imediatamente.

Quando usar: Comandos momentâneos, sinalização de estados transitórios, saídas que devem espelhar condição lógica sem memorização. Não utilizar para partida de motor sem intertravamento SR/RS.

Sintaxe e Parâmetros: Operando (BOOL): Saída %Q, memória M ou bit de DB. Em S7-1200/1500, bobinas em OBs de alta prioridade podem sobrepor saídas de OB1 — respeite hierarquia de OBs.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Bobina AQUI]

```

      Condicao_OK
-----| |----- ( ) Lamp_Ready

```

STL (Statement List):

```

A      "Condicao_OK"
=      "Lamp_Ready"

```

Set (S) e Reset (R) — Bobinas de Memorização

Nome e Objetivo: Instruções de coil que alteram o estado do operando para TRUE (S) ou FALSE (R) quando o RLO = 1, mantendo o estado entre ciclos. Ordem de varredura define prioridade em conflito.

Quando usar: Sequências simples, alarmes latched, reset de falhas. Em STL clássico S7-300/400, S e R na mesma rede: o último processado vence. Para prioridade explícita, use blocos RS/SR.

Sintaxe e Parâmetros: Operando (BOOL): Bit a memorizar. S: set dominante se escrito por último. R: reset dominante se escrito por último. Não usar S/R no mesmo operando em redes paralelas sem análise de prioridade.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Set/Reset AQUI]

```

      Btn_Start
-----| |----- (S) Motor_Run

      Btn_Stop
-----| |----- (R) Motor_Run

```

STL (Statement List):

```

// Reset dominante: R após S
A      "Btn_Start"
S      "Motor_Run"
A      "Btn_Stop"
R      "Motor_Run"

```

1.2 Memorização de Estado e Flip-Flops (SR / RS)

SR (Set Dominant) e RS (Reset Dominant)

Nome e Objetivo: Blocos de flip-flop para memorização de estado lógico. SR prioriza Set quando S1 e R estão simultaneamente TRUE ($Q = 1$). RS prioriza Reset ($Q = 0$). Garantem comportamento determinístico em comando concorrente.

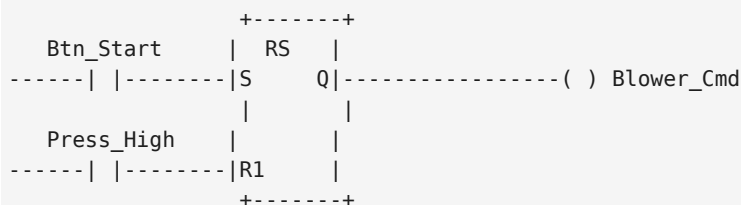
Quando usar: Partida/parada de motores, válvulas ON/OFF, intertravamentos de processo. RS obrigatório quando parada de emergência ou condição de segurança não pode ser sobreposta por partida. SR apenas quando Set deve prevalecer em conflito (ex.: modo automático forçado).

Sintaxe e Parâmetros: S1/S (BOOL): Condição de Set. R/R1 (BOOL): Condição de Reset. Q (BOOL): Estado memorizado. Instanciar como multi-instância em FB ou DB dedicado tipo SR/RS.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RS AQUI]



STL (Statement List):

```
// Reset dominante via ordem STL
A   "Btn_Start"
S   "Blower_Cmd"
A   "Press_High"
R   "Blower_Cmd"
```

```
// Ou via CALL do bloco IEC
CALL "RS_Blower", "DB_RS_Blower"
    SET := "Btn_Start"
    RESET1 := "Press_High"
    Q := "Blower_Cmd"
```

SR — Set Dominant Flip-Flop

Nome e Objetivo: Variante com prioridade Set: se S1 e R simultâneos, $Q = 1$. Utilizar apenas quando partida/comando ON deve prevalecer sobre parada em conflito de sinais.

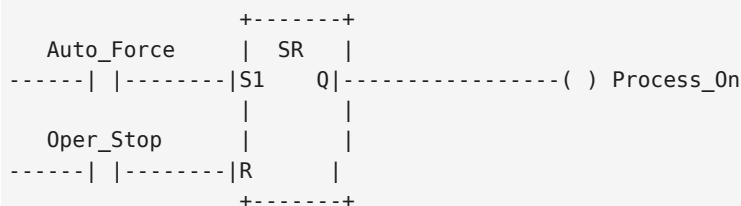
Quando usar: Modo automático forçado por supervisor, partida prioritária em processo contínuo onde parada momentânea de R é aceitável ser ignorada — raro; documentar FMEA.

Sintaxe e Parâmetros: S1 (BOOL): Set. R (BOOL): Reset. Q (BOOL): Saída. Bloco SR em paleta Bit logic.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SR AQUI]



STL (Statement List):

```
CALL  "SR_Process", "DB_SR_Process"
      S1 := "Auto_Force"
      R  := "Oper_Stop"
      Q  := "Process_On"
```

1.3 Operações de Detecção de Borda (P_TRIG / N_TRIG / R_TRIG / F_TRIG)

P_TRIG (Positive Trigger) / FP em STL

Nome e Objetivo: Emite pulso TRUE por exatamente 1 ciclo de scan na transição 0→1 do sinal monitorado. R_TRIG é variante em bloco com saída Q e entrada CLK.

Quando usar: Disparo único de requisições (REQ em GET/PUT), incremento de contadores de evento, registro de alarmes sem repetição cíclica, handshake de comunicação Profinet.

Sintaxe e Parâmetros: CLK (BOOL): Sinal monitorado. M_BIT (BOOL): Memória de borda — exclusiva, estática em FB. Q (BOOL): Pulso de 1 scan. Em STL: FP "Edge_Mem" após A do sinal.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO P_TRIG AQUI]

```

      Inv_Fault          +-----+
-----| |-----+-----| P_TRIG |-----+-----+ ( ) Log_Trigger
                        |         |
"Inst_DB".Edge_Mem_InvFault----| M_BIT |
                        +-----+
```

STL (Statement List):

```
A      "Inv_Fault"
FP      "Inst_DB".Edge_Mem_InvFault
=      "Log_Trigger"
```

N_TRIG (Negative Trigger) / FN em STL

Nome e Objetivo: Pulso de 1 ciclo na transição 1→0. F_TRIG é equivalente em bloco.

Quando usar: Detecção de fim de ciclo, liberação de intertravamento após retorno de sensor, contagem de peças na saída da zona de detecção.

Sintaxe e Parâmetros: Mesmos parâmetros do P_TRIG. FN em STL requer memória de borda dedicada.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO N_TRIG AQUI]

```

      Sens_Peca          +-----+
-----| |-----+-----| N_TRIG |-----+-----+ ( ) Peca_Saida
                        |         |
"DB_Prod".Edge_Mem_Sens-----| M_BIT |
                        +-----+
```

STL (Statement List):

```
A      "Sens_Peca"
FN      "DB_Prod".Edge_Mem_Sens
=      "Peca_Saida"
```

R_TRIG / F_TRIG (Blocos IEC)

Nome e Objetivo: Blocos encapsulados de detecção de borda positiva (R_TRIG) e negativa (F_TRIG) com interface padronizada EN/CLK/Q.

Quando usar: FBs reutilizáveis, bibliotecas corporativas BLBW, padronização de código em equipes multidisciplinares.

Sintaxe e Parâmetros: CLK (BOOL): Entrada. Q (BOOL): Saída pulsada. Memória interna no DB de instância — não expor M_BIT externamente.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO R_TRIG AQUI]

```

      +-----+
Start_Pulse | R_TRIG |
-----| |-----|CLK   Q|----- ( ) Step_Advance
      +-----+

```

STL (Statement List):

```

CALL  "R_TRIG_Step", "DB_R_TRIG_Step"
      CLK := "Start_Pulse"
      Q   := "Step_Advance"

```


CAPÍTULO 2: Timer Operations (IEC Timers)

2.1 TON (Timer On-Delay)

TON — Generate On-Delay

Nome e Objetivo: Temporizador IEC com retardo na ativação. Q = TRUE somente após IN permanecer TRUE durante PT. Perda de IN zera ET e Q imediatamente.

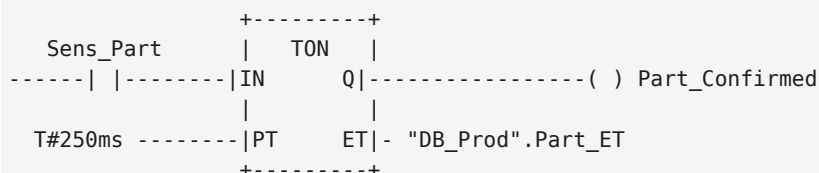
Quando usar: Debounce de sensores, sequenciamento de partidas escalonadas, retardo pós-rearme de emergência, confirmação de presença de peça.

Sintaxe e Parâmetros: IN (BOOL): Habilitação. PT (TIME): Preset T#250ms, T#5s, etc. Q (BOOL): Saída temporizada. ET (TIME): Tempo decorrido. DB: IEC_TIMER ou instância em FB.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TON AQUI]



STL (Statement List):

```
CALL "TON_Part_Instance"
  IN := "Sens_Part"
  PT := T#250MS
  Q := "Part_Confirmed"
  ET := "DB_Prod".Part_ET
```

2.2 TOF (Timer Off-Delay)

TOF — Generate Off-Delay

Nome e Objetivo: Q assume TRUE instantaneamente com IN = TRUE. Ao cair IN, inicia contagem de PT antes de Q = FALSE. Retorno de IN durante contagem cancela e reinicia.

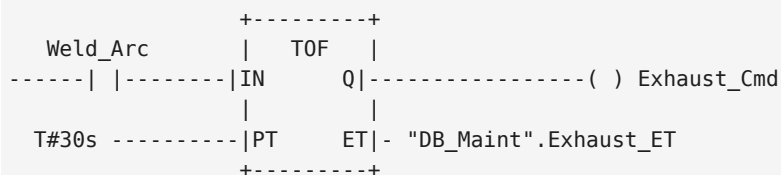
Quando usar: Ventilação pós-processo, lubrificação residual, extração de fumos após solda, despressurização com dreno temporizado.

Sintaxe e Parâmetros: IN, PT, Q, ET — idênticos ao TON. Tipo de timer: IEC_TIMER.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TOF AQUI]



STL (Statement List):

```
CALL "TOF_Exhaust_Instance"
  IN := "Weld_Arc"
```

```
PT := T#30S
Q  := "Exhaust_Cmd"
ET := "DB_Maint".Exhaust_ET
```

2.3 TP (Timer Pulse)

TP — Generate Pulse

Nome e Objetivo: Gera pulso de duração fixa PT na borda de subida de IN. Q permanece TRUE por PT independente do estado de IN após disparo. Nova borda só é aceita após término do pulso.

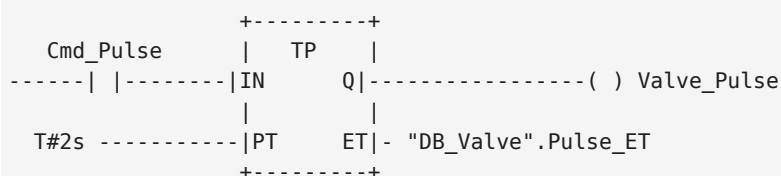
Quando usar: Impulso de válvula proporcional, blink de alarme, pulso de reset remoto, temporização de acionamento de relé monostável.

Sintaxe e Parâmetros: IN (BOOL): Flanco de disparo. PT (TIME): Duração do pulso. Q (BOOL): Saída pulsada. ET: tempo restante do pulso.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TP AQUI]



STL (Statement List):

```
CALL "TP_Valve_Instance"
  IN := "Cmd_Pulse"
  PT := T#2S
  Q  := "Valve_Pulse"
  ET := "DB_Valve".Pulse_ET
```

2.4 TONR (Time Accumulator)

TONR — Time Accumulator

Nome e Objetivo: Acumula tempo de IN = TRUE entre ciclos. R (BOOL) zera ET e Q. Diferente do TON, perda momentânea de IN preserva tempo acumulado.

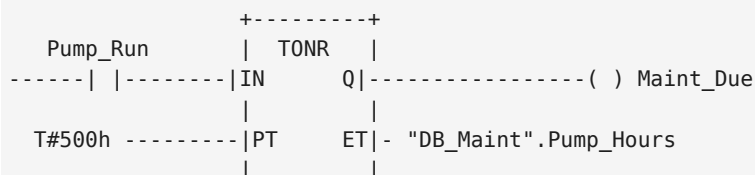
Quando usar: Horímetro de equipamento, manutenção preventiva por tempo de operação, controle de tempo máximo de ciclo acumulado, billing de tempo de bomba.

Sintaxe e Parâmetros: IN (BOOL): Contagem ativa. PT (TIME): Tempo alvo acumulado. R (BOOL): Reset. Q (BOOL): TRUE quando ET >= PT. ET (TIME): Tempo total acumulado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TONR AQUI]



Reset_Hours		R	
-----	-----		
		+-----+	

STL (Statement List):

```
CALL  "TONR_Pump_Hours"
  IN  := "Pump_Run"
  PT  := T#500H
  R    := "Reset_Hours"
  Q    := "Maint_Due"
  ET  := "DB_Maint".Pump_Hours
```

CAPÍTULO 3: Counter Operations (IEC Counters)

3.1 CTU (Count Up)

CTU — Count Up

Nome e Objetivo: Contador incremental IEC. Incrementa CV em borda de subida de CU quando R = FALSE. Q = TRUE quando CV >= PV.

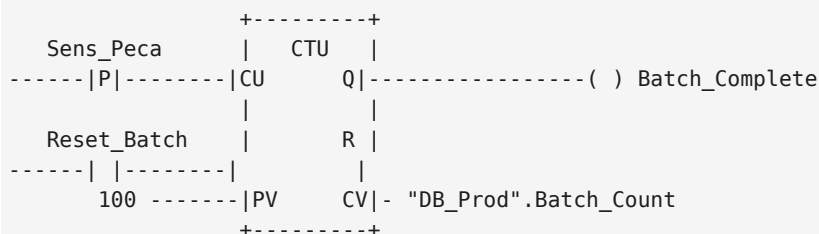
Quando usar: Contagem de peças produzidas, batching, limite de ciclos para manutenção, controle de paletização.

Sintaxe e Parâmetros: CU (BOOL): Count Up — borda de subida. R (BOOL): Reset CV para 0. PV (INT/DINT): Preset. CV (INT/DINT): Valor atual. Q (BOOL): CV >= PV.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTU AQUI]



STL (Statement List):

```

CALL  "CTU_Batch"
CU   := "Sens_Peca"
R    := "Reset_Batch"
PV   := 100
Q    := "Batch_Complete"
CV   := "DB_Prod".Batch_Count
  
```

3.2 CTD (Count Down)

CTD — Count Down

Nome e Objetivo: Contador decremental. CD em borda de subida decrementa CV. LD carrega PV em CV. Q = TRUE quando CV <= 0.

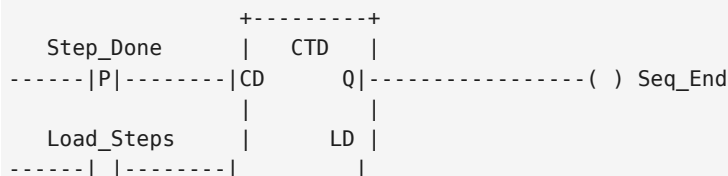
Quando usar: Contagem regressiva de etapas, dispensação de N doses, sequenciador com número fixo de passos restantes.

Sintaxe e Parâmetros: CD (BOOL): Count Down. LD (BOOL): Load — carrega PV em CV. PV, CV, Q conforme CTU. Q ativo em CV = 0.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTD AQUI]



```

10 -----|PV      CV|- "DB_Seq".Steps_Left
          +-----+

```

STL (Statement List):

```

CALL  "CTD_Sequence"
  CD := "Step_Done"
  LD := "Load_Steps"
  PV := 10
  Q  := "Seq_End"
  CV := "DB_Seq".Steps_Left

```

3.3 CTUD (Count Up/Down)

CTUD — Count Up/Down

Nome e Objetivo: Contador bidirecional. CU incrementa, CD decrementa. QU e QD indicam $CV \geq PV$ e $CV \leq 0$ respectivamente. R zera CV.

Quando usar: Controle de estoque em silos, posicionamento incremental, filas de buffer com entrada/saída simultânea.

Sintaxe e Parâmetros: CU, CD, R, LD, PV, CV, QU, QD. Prioridade de CU e CD simultâneos: conforme manual IEC — verificar versão de firmware.

Exemplo Prático de Aplicação:**LAD (Ladder):**

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTUD AQUI]

```

          +-----+
Entry_Sens | CTUD |
-----|P|-----|CU    QU|----- ( ) Silo_Full
          |      |
Exit_Sens  |      |
-----|P|-----|CD    QD|----- ( ) Silo_Empty
          |      |
Reset_Level|      |R      |
-----| |-----|      |
1000 -----|PV    CV|- "DB_Silo".Level
          +-----+

```

STL (Statement List):

```

CALL  "CTUD_Silo_Level"
  CU := "Entry_Sens"
  CD := "Exit_Sens"
  R  := "Reset_Level"
  PV := 1000
  QU := "Silo_Full"
  QD := "Silo_Empty"
  CV := "DB_Silo".Level

```

CAPÍTULO 4: Comparator Operations

4.1 Operações Matemáticas Relacionais (==, <>, >=, <=, >, <)

CMP == / CMP <> / CMP >= / CMP <= / CMP > / CMP <

Nome e Objetivo: Comparação aritmética entre dois operandos do mesmo tipo numérico ou tempo. Resultado BOOL na saída do box ou como condição de contato em LAD.

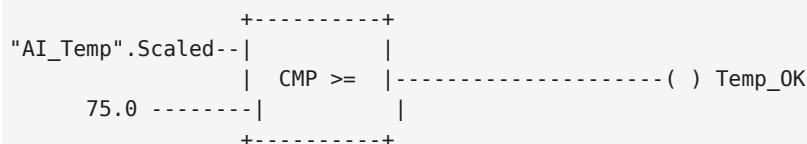
Quando usar: Setpoint de temperatura, comparação de posição encoder, verificação de receita, intertravamento por faixa de velocidade.

Sintaxe e Parâmetros: IN1, IN2: Operandos comparáveis (INT, DINT, REAL, TIME). Tipos devem coincidir ou usar CONVERT prévio. Saída BOOL integrada ao RLO em LAD.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CMP >= AQUI]



STL (Statement List):

```

L    "AI_Temp".Scaled
>=R  75.0
=    "Temp_OK"
  
```

CMP == (Equal) e CMP <> (Not Equal)

Nome e Objetivo: Igualdade (==) ou diferença (<>) exata entre operandos. Cuidado com REAL: comparar com tolerância via IN_Range em vez de == direto.

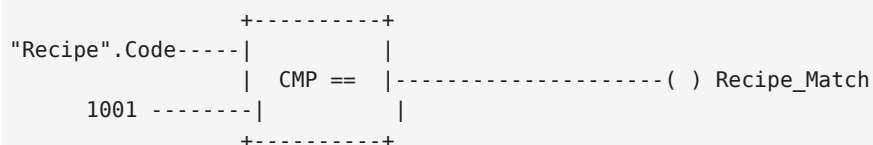
Quando usar: Verificação de código de receita, estado discreto numérico, comparação de posição em passo fixo.

Sintaxe e Parâmetros: IN1, IN2: tipos homogêneos. Para REAL use ABS(IN1-IN2) < Epsilon via CALCULATE.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CMP == AQUI]



STL (Statement List):

```

L    "Recipe".Code
==I  1001
=    "Recipe_Match"
  
```

4.2 Checagem de Limites (IN_Range / OUT_Range)

IN_Range / OUT_Range

Nome e Objetivo: IN_Range: TRUE se $MIN \leq IN \leq MAX$. OUT_Range: TRUE se $IN < MIN$ ou $IN > MAX$. Simplifica faixas sem dupla comparação.

Quando usar: Validação de sinal analógico dentro de faixa operacional, detecção de sensor quebrado (4-20mA), tolerância de processo em estação SanBox BLBW.

Sintaxe e Parâmetros: IN (numeric): Valor avaliado. MIN, MAX: Limites inclusivos (IN_Range). Tipos homogêneos REAL recomendado para analógicos.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO IN_Range AQUI]

```

      +-----+
      | "AI_Press".Value-|
      |   | IN_Range |----- ( ) Press_Valid
      | 2.0 -----|
      | 10.0 -----|
      +-----+
  
```

STL (Statement List):

```

CALL  "IN_Range_Press"
      IN  := "AI_Press".Value
      MIN := 2.0
      MAX := 10.0
      OUT := "Press_Valid"
  
```

CAPÍTULO 5: Math Functions

5.1 Cálculos Aritméticos Básicos (ADD, SUB, MUL, DIV)

ADD / SUB / MUL / DIV

Nome e Objetivo: Operações aritméticas IEC com EN/ENO. DIV com proteção: ENO = FALSE em divisão por zero — sempre monitorar ENO em cálculos críticos.

Quando usar: Cálculo de vazão, balanço massa, offset de setpoint, conversão de unidades em runtime.

Sintaxe e Parâmetros: EN (BOOL): Habilitação. IN1, IN2: Operandos. OUT: Resultado. ENO (BOOL): Sem erro. Tipos: INT, DINT, REAL — resultado conforme tipo maior.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ADD AQUI]

```

      +-----+
      |          |
      | Calc_Enable | ADD |
      |-----| |-----| EN      ENO |-----+-----+ ( ) Calc_OK
      | "Flow_A".Value |
      | "Flow_B".Value | IN1      OUT |--- "Total_Flow".Value
      |          | IN2      |
      |          |
      +-----+
  
```

STL (Statement List):

```

A      "Calc_Enable"
CALL   "ADD_Flow"
      IN1 := "Flow_A".Value
      IN2 := "Flow_B".Value
      OUT := "Total_Flow".Value
      ENO => "Calc_OK"
  
```

DIV — Divide (com monitoramento ENO)

Nome e Objetivo: Divisão com detecção de divisor zero. ENO=FALSE interrompe cadeia EN/ENO — downstream não executa.

Quando usar: Cálculo de média, razão de vazões, conversão de unidade. Sempre inserir ramo de fallback quando ENO=FALSE.

Sintaxe e Parâmetros: IN1/IN2/OUT. Tipos REAL recomendados para analógicos. INT division trunca — usar REAL.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO DIV AQUI]

```

      +-----+
      |          |
      | Calc_En    | DIV |
      |-----| |-----| EN      ENO |---+-----+ ( ) Div_OK
      | "Flow".Total |
      | "Flow".Hours | IN1      OUT |---|--- "Flow".Avg
      |          | IN2      |
      |          |
      +-----+
                        +-----+ ( ) Div_Fault
  
```

STL (Statement List):


```

A      "Calc_En"
CALL  "DIV_AvgFlow"
      IN1 := "Flow".Total
      IN2 := "Flow".Hours
      OUT := "Flow".Avg
      ENO => "Div_OK"
AN     "Div_OK"
=      "Div_Fault"

```

5.2 Seleção de Extremos e Limitação (MIN, MAX, LIMIT)

MIN / MAX / LIMIT

Nome e Objetivo: MIN: menor de IN1/IN2. MAX: maior. LIMIT: restringe IN entre MN e MX (MN se $IN < MN$, MX se $IN > MX$, senão IN).

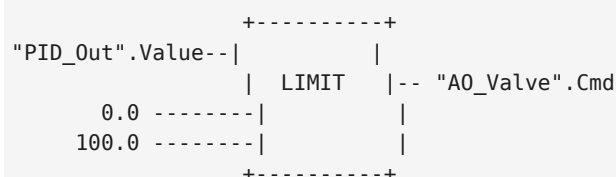
Quando usar: Saturação de comando analógico, seleção de menor pressão entre redundância, clamping de PID output, proteção de atuador.

Sintaxe e Parâmetros: IN1, IN2 (MIN/MAX). IN, MN, MX (LIMIT). OUT: resultado. Tipos REAL para analógicos.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO LIMIT AQUI]



STL (Statement List):

```

CALL  "LIMIT_Valve"
      IN := "PID_Out".Value
      MN := 0.0
      MX := 100.0
      OUT := "AO_Valve".Cmd

```

5.3 Funções Trigonométricas e Exponenciais Avançadas

SIN / COS / TAN / SQRT / EXPT / LN / EXP

Nome e Objetivo: Funções matemáticas em REAL. Entrada e saída em radianos para trigonométricas. SQRT de valor negativo gera ENO = FALSE.

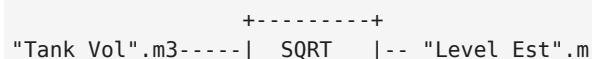
Quando usar: Cinemática de braço robótico, cálculo de coordenadas em esteira circular, escalonamento não linear, dosing exponencial.

Sintaxe e Parâmetros: IN (REAL): Operando. OUT (REAL): Resultado. ENO: status. CALCULATE disponível para expressões compostas.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SQRT AQUI]



+-----+

STL (Statement List):

```
CALL  "SQRT_Level"  
  IN   := "Tank_Vol".m3  
  OUT  := "Level_Est".m
```

CAPÍTULO 6: Move Operations

6.1 Transferência de Bloco Escalar (MOVE)

MOVE — Move Value

Nome e Objetivo: Cópia de valor entre tags de tipos compatíveis. Suporta BOOL até estruturas e VARIANT conforme versão. Substituto moderno de BLK MOV e transferências STL.

Quando usar: Cópia de setpoint recebido via comunicação, transferência de estrutura de receita, inicialização de parâmetros de FB.

Sintaxe e Parâmetros: EN, ENO. IN: origem. OUT: destino. Tipos devem ser atribuíveis — usar CONVERT se necessário.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MOVE AQUI]

```

      +-----+
      | Update_Recipe | MOVE |
      |-----| |-----| EN      ENO |
      | "HMI".Setpoint | IN      OUT | -- "DB_Recipe".Setpoint
      +-----+
  
```

STL (Statement List):

```

A      "Update_Recipe"
CALL   "MOVE_Setpoint"
      IN  := "HMI".Setpoint
      OUT := "DB_Recipe".Setpoint
  
```

6.2 Manipulação em Lote e Serialização (MOVE_BLK, FILL_BLK, SWAP)

MOVE_BLK / FILL_BLK / SWAP

Nome e Objetivo: MOVE_BLK: copia N elementos de array. FILL_BLK: preenche array com valor constante. SWAP: inverte ordem de bytes (endianness) em WORD/DWORD.

Quando usar: Buffer de comunicação, cópia de zona de DB, inicialização de arrays de tendência, conversão Modbus RTU (byte swap).

Sintaxe e Parâmetros: MOVE_BLK: SRC, DST, COUNT, INDEX. FILL_BLK: SRC (valor), DST, COUNT. SWAP: IN/OUT WORD/DWORD.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MOVE_BLK AQUI]

```

      +-----+
      | Copy_Enable | MOVE_BLK | | |
      |-----| |-----| EN      ENO |
      | "Src_Array" |-----| |
      | "Dst_Array" |-----| |
      | 20 |-----| COUNT |
      +-----+
  
```

STL (Statement List):

```

CALL   "MOVE_BLK_Data"
      SRC_ARRAY := "Src_Array"
  
```

```
DST_ARRAY := "Dst_Array"
COUNT   := 20
```

Serialize / Deserialize

Nome e Objetivo: Serializa estrutura tipada em array de BYTE para transmissão. Deserialize reconstrói estrutura a partir do buffer.

Quando usar: Telemetria SanBox BLBW, envio de struct via PUT/GET, logging compacto em CPU sem cartão SD.

Sintaxe e Parâmetros: SRC/DST: VARIANT ou struct. POS: offset no buffer. BUFFER: ARRAY OF BYTE.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Serialize AQUI]

```

      Tx_Enable      +-----+
                    | Serialize |
-----| |-----|EN      ENO|
"DB_Telemetry"----|          |
"Tx_Buffer"-----|          |
                    +-----+
```

STL (Statement List):

```
CALL "Serialize_Telemetry"
SRC   := "DB_Telemetry"
BUFFER := "Tx_Buffer"
POS   := 0
```

CAPÍTULO 7: Conversion Operations

7.1 Conversão Explícita de Cast (CONVERT)

CONVERT

Nome e Objetivo: Conversão tipada explícita entre tipos numéricos (INT↔REAL↔DINT↔DWORD). Overflow gera ENO = FALSE em alguns casos — validar.

Quando usar: Processamento de AI (INT para REAL), preparação de operandos para CMP e math, integração com drives (DWORD).

Sintaxe e Parâmetros: IN: tipo origem. OUT: tipo destino declarado no box. ENO: status.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CONVERT AQUI]

```

      +-----+
"AI_Raw".Int-----|  CONVERT  |-- "AI_Raw".Real
      +-----+

```

STL (Statement List):

```

CALL  "CONVERT_AI"
  IN   := "AI_Raw".Int
  OUT  := "AI_Raw".Real

```

7.2 Arredondamento e Truncamento (ROUND, CEIL, FLOOR, TRUNC)

ROUND / CEIL / FLOOR / TRUNC

Nome e Objetivo: ROUND: arredondamento matemático para inteiro. CEIL: próximo inteiro superior. FLOOR: próximo inferior. TRUNC: descarta parte fracionária.

Quando usar: Exibição em IHM sem decimal, cálculo de quantidade inteira de embalagens, indexação de array por REAL.

Sintaxe e Parâmetros: IN (REAL/LREAL). OUT (INT/DINT conforme bloco).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ROUND AQUI]

```

      +-----+
"Dose_Kg".Real----|  ROUND  |-- "Dose_Kg".Int
      +-----+

```

STL (Statement List):

```

CALL  "ROUND_Dose"
  IN   := "Dose_Kg".Real
  OUT  := "Dose_Kg".Int

```

CEIL / FLOOR — Limites Inteiros Superior/Inferior

Nome e Objetivo: CEIL: menor inteiro $\geq$ IN. FLOOR: maior inteiro $\leq$ IN. Diferem de ROUND e TRUNC em valores negativos e .5 exatos.

Quando usar: Dimensionamento de lotes (CEIL para garantir cobertura), cálculo de capacidade mínima de recipientes.

Sintaxe e Parâmetros: IN: REAL/LREAL. OUT: INT/DINT.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CEIL AQUI]

```

      +-----+
"Batches".Real----| CEIL  |-- "Batches".Count
      +-----+
  
```

STL (Statement List):

```

CALL  "CEIL_Batches"
    IN  := "Batches".Real
    OUT := "Batches".Count
  
```

7.3 Escalonamento de Sinais Analógicos (SCALE_X, NORM_X)

SCALE_X / NORM_X

Nome e Objetivo: NORM_X: normaliza entrada de faixa RAW (ex. 0-27648) para 0.0-1.0. SCALE_X: aplica ganho e offset de 0-1 para engenharia (0-10 bar). Par inverso para escrita em AO.

Quando usar: Padronização de tratamento AI/AO em biblioteca BLBW, eliminação de fórmulas manuais em cada tag analógica.

Sintaxe e Parâmetros: NORM_X: MIN, VALUE, MAX → OUT 0..1. SCALE_X: MIN, VALUE(0..1), MAX → engenharia.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SCALE_X AQUI]

```

      +-----+
"AI_Raw".Value----| NORM_X |-- "AI_Norm".Pct
    0  -----|          |
 27648 -----|          |
      +-----+
  
```

```

      +-----+
"AI_Norm".Pct----| SCALE_X |-- "AI_Press".Bar
    0.0 -----|          |
   10.0 -----|          |
      +-----+
  
```

STL (Statement List):

```

CALL  "NORM_X_AI"
    MIN  := 0
    VALUE := "AI_Raw".Value
    MAX  := 27648
    OUT  := "AI_Norm".Pct

CALL  "SCALE_X_Press"
  
```

```
MIN    := 0.0  
VALUE  := "AI_Norm".Pct  
MAX    := 10.0  
OUT    := "AI_Press".Bar
```

CAPÍTULO 8: Program Control Operations

8.1 Saltos e Roteamento Lógico (JMP, JMPN, LABEL, SWTCH)

JMP / JMPN / LABEL

Nome e Objetivo: JMP: salta para LABEL se RLO=1. JMPN: salta se RLO=0. LABEL: destino do salto. Código entre JMP e LABEL não é processado quando salto ativo.

Quando usar: Otimização de scan em OB1 extensa, bypass de diagnóstico, desabilitação condicional de redes STL. Usar com moderação — prejudica rastreabilidade.

Sintaxe e Parâmetros: LABEL: identificador único na rede. JMP/JMPN: coil-style em LAD. Apenas dentro do mesmo OB/FC.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO JMP AQUI]

      Skip_Diag
-----| |----- (JMP) Label_Prod

      // Redes de diagnóstico omitidas quando Skip_Diag = TRUE

                        LABEL
Label_Prod:
Process_Run
-----| |----- ( ) Line_Run
```

STL (Statement List):

```
A      "Skip_Diag"
JMP    Label_Prod
// ... diagnóstico ...
Label_Prod: A      "Process_Run"
=      "Line_Run"
```

SWTCH (Jump Distributor)

Nome e Objetivo: Distribuidor de salto indexado. K (INT) seleciona qual saída CASE é ativada, roteando fluxo para múltiplos LABELs.

Quando usar: Sequenciadores em STL puro, máquinas de estados em FC sem GRAPH, seleção de rotina por código de receita.

Sintaxe e Parâmetros: K (INT): índice 1..n. Entradas CASE1..CASEn com LABEL associado.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SWTCH AQUI]

      +-----+
"Recipe".ID-----|  SWTCH  |
                  | K:=ID   |
                  | CASE1   | ----> Label_R1
                  | CASE2   | ----> Label_R2
                  +-----+
```

STL (Statement List):

```
L      "Recipe".ID
```



```
SWTCH  
CASE1 Label_R1  
CASE2 Label_R2
```

8.2 Finalização de Bloco (RET)

RET — Return

Nome e Objetivo: Encerra execução do OB/FC/Fb atual imediatamente. Redes subsequentes não são processadas neste ciclo.

Quando usar: FC de comunicação com abort em erro, proteção contra execução sem habilitação, saída antecipada em diagnóstico.

Sintaxe e Parâmetros: RET como coil quando RLO=1. Em STL: RET condicionado ou incondicional.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RET AQUI]

```
      Comm_Error  
-----| |----- (RET)
```

STL (Statement List):

```
A      "Comm_Error"  
RET
```

CAPÍTULO 9: Word Logic Operations

9.1 Mascaramento a Nível de Palavra (AND, OR, XOR)

AND / OR / XOR (Word)

Nome e Objetivo: Operação bit a bit entre WORD/DWORD. AND: máscara de bits ativos. OR: união. XOR: comparação e toggle.

Quando usar: Máscara de status de drive em DWORD, extração de nibble, diagnóstico de palavra de falha Profinet, reset seletivo de bits.

Sintaxe e Parâmetros: IN1, IN2 (WORD/DWORD). OUT: resultado mesma largura.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO AND Word AQUI]

```

      +-----+
"Drive".Status---|   AND   |-- "Drive".Masked
16#00FF -----|         |
      +-----+
```

STL (Statement List):

```

L   "Drive".Status
L   16#00FF
AND
T   "Drive".Masked
```

INVERT — Ones Complement (Word)

Nome e Objetivo: Inverte todos os bits da palavra (complemento de um). OUT = NOT(IN) bit a bit.

Quando usar: Geração de máscara invertida, protocolo serial, teste de redundância XOR.

Sintaxe e Parâmetros: IN/OUT: WORD/DWORD.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO INVERT AQUI]

```

      +-----+
"Mask".Template---| INVERT |-- "Mask".Inverted
      +-----+
```

STL (Statement List):

```

L   "Mask".Template
INV_I
T   "Mask".Inverted
```

9.2 Manipulação Estrutural (ENCO, DECO, SEL, MUX, DEMUX)

ENCO / DECO

Nome e Objetivo: ENCO: converte posição de bit TRUE em INT (encoder one-hot). DECO: converte INT em word com bit correspondente em TRUE.

Quando usar: Seleção de canal ativo, decodificador de posição em matriz de válvulas, interface com hardware one-hot.

Sintaxe e Parâmetros: ENCO: IN (WORD), OUT (INT bit position). DECO: IN (INT), OUT (WORD).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ENCO AQUI]

```

      +-----+
"Valve_Mask".Word-|  ENCO  |-- "Valve_Active".Idx
      +-----+

```

STL (Statement List):

```

CALL  "ENCO_Valve"
      IN   := "Valve_Mask".Word
      OUT  := "Valve_Active".Idx

```

SEL / MUX / DEMUX

Nome e Objetivo: SEL: se G=0 OUT=IN0, se G=1 OUT=IN1. MUX: seleciona entre N entradas por índice K. DEMUX: distribui entrada para uma de N saídas por K.

Quando usar: Seletor manual/auto de setpoint, multiplexação de sensores redundantes, roteamento de comando para um de vários atuadores.

Sintaxe e Parâmetros: SEL: G, IN0, IN1. MUX: K (INT), IN0..INn. DEMUX: K, IN, OUT0..OUTn.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MUX AQUI]

```

      +-----+
"Mode".Manual-----|  MUX  |-- "Setpoint".Active
"HMI".SP_Man-----|         |
"DCS".SP_Auto-----|         |
"Mode".SelIdx-----|  K     |
      +-----+

```

STL (Statement List):

```

CALL  "MUX_Setpoint"
      K    := "Mode".SelIdx
      IN0  := "HMI".SP_Man
      IN1  := "DCS".SP_Auto
      OUT  := "Setpoint".Active

```

CAPÍTULO 10: Shift and Rotate Operations

10.1 Deslocamento de Bits (SHR, SHL)

SHR / SHL

Nome e Objetivo: SHR: desloca bits para direita, bits baixos perdem, entrada N define quantidade. SHL: desloca à esquerda. Útil para divisão/multiplicação por potência de 2.

Quando usar: Serialização manual de bits, conversão de formato de encoder, alinhamento de dados em buffer, otimização aritmética em INT.

Sintaxe e Parâmetros: IN (WORD/DWORD). N (INT): posições. OUT: resultado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SHR AQUI]

```

      +-----+
"Reg".Data-----|  SHR  |-- "Reg".Shifted
      2 -----|  N    |
      +-----+
  
```

STL (Statement List):

```

CALL  "SHR_Reg"
  IN   := "Reg".Data
  N    := 2
  OUT  := "Reg".Shifted
  
```

SHL — Shift Left

Nome e Objetivo: Deslocamento à esquerda N posições. Bits altos perdem. Equivalente a multiplicação por 2^N para valores sem sinal.

Quando usar: Multiplicação rápida por potência de 2, montagem de word a partir de nibbles, endereçamento em word de comando.

Sintaxe e Parâmetros: IN (WORD/DWORD), N (INT), OUT.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SHL AQUI]

```

      +-----+
"Val".Word-----|  SHL  |-- "Val".X2
      1 -----|  N    |
      +-----+
  
```

STL (Statement List):

```

CALL  "SHL_Val"
  IN   := "Val".Word
  N    := 1
  OUT  := "Val".X2
  
```

10.2 Rotação de Registradores (ROR, ROL)

ROR / ROL

Nome e Objetivo: Rotação circular de bits — bits que saem reentram do lado oposto. Preserva todos os bits sem perda.

Quando usar: CRC bit-a-bit, criptografia leve de token, simulação de registrador de deslocamento em hardware, algoritmos de hash industrial.

Sintaxe e Parâmetros: IN (WORD/DWORD). N (INT): posições de rotação. OUT: resultado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ROR AQUI]

```

      +-----+
"Token".Word-----|  ROR  |-- "Token".Rotated
      1 -----|  N    |
      +-----+
  
```

STL (Statement List):

```

CALL  "ROR_Token"
  IN   := "Token".Word
  N     := 1
  OUT  := "Token".Rotated
  
```

CAPÍTULO 1 (continuação): Bit Logic Operations

1.4 Inversão de RLO, Bobinas Especiais e Contatos de Borda

Invert RLO — |NOT|

Nome e Objetivo: Inverte o Result of Logic Operation (RLO) atual sem alterar o operando de entrada. O estado do bit monitorado permanece intacto; apenas a corrente lógica é negada.

Quando usar: Inversão de cadeia lógica sem criar contato NF adicional, simplificação de redes com múltiplas negações, preparação de condição para bloco que requer RLO invertido.

Sintaxe e Parâmetros: Sem operando associado. Atua sobre o RLO acumulado na rede. Em STL: NOT após A/AN. Em LAD: elemento |NOT| em série.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Invert RLO AQUI]

```

Cond_A
-----| |-----|NOT|-----+----- ( ) Output_Inv
                                   |
Cond_B
-----| |-----+-----

```

STL (Statement List):

```

A      "Cond_A"
NOT
A      "Cond_B"
=      "Output_Inv"

```

Bobina Negada — -(/)-

Nome e Objetivo: Atribui o complemento do RLO ao operando: $Q = \text{NOT}(\text{RLO})$. Quando a cadeia lógica conduz, o operando é resetado; quando não conduz, o operando é setado.

Quando usar: Sinalização invertida de estado (lâmpada de 'não pronto'), interface com hardware active-low, lógica de falha silenciosa onde ausência de condição = alarme.

Sintaxe e Parâmetros: Operando (BOOL): Saída ou memória. Comportamento oposto à bobina -()-.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Bobina Negada AQUI]

```

System_Fault
-----| |------(/) Lamp_OK

```

STL (Statement List):

```

A      "System_Fault"
NOT
=      "Lamp_OK"

```

Contato de Borda Positiva — |P| e Bobina P — -(P)-

Nome e Objetivo: |P|: conduz por 1 scan na transição 0→1 do operando. -(P)-: seta o operando por 1 scan na borda de subida do RLO. Equivalente funcional a P_TRIG/FP sem bloco dedicado.

Quando usar: Incremento de contador em rede única, latch momentâneo de comando, alternativa compacta quando não há FB disponível.

Sintaxe e Parâmetros: |P|: operando monitorado com memória interna implícita no compilador. -(P)-: operando de saída pulsada. Preferir R_TRIG em código reutilizável.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato P AQUI]

```

      Btn_Count
-----|P|----- ( ) Count_Pulse

```

STL (Statement List):

```

A      "Btn_Count"
FP     "DB_Edge".Btn_Mem
=      "Count_Pulse"

```

Contato de Borda Negativa — |N| e Bobina N — -(N)-

Nome e Objetivo: |N|: conduz por 1 scan na transição 1→0. -(N)-: reset pulsado na borda de descida do RLO.

Quando usar: Detecção de desacionamento de sensor, contagem de retirada de peça, trigger de log ao desligar equipamento.

Sintaxe e Parâmetros: Equivalente a FN/F_TRIG. Memória de borda obrigatória em STL.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato N AQUI]

```

      Sens_Out
-----|N|----- ( ) Piece_Left_Zone

```

STL (Statement List):

```

A      "Sens_Out"
FN     "DB_Edge".Sens_Mem
=      "Piece_Left_Zone"

```

1.5 Operações em Campo de Bits (SET_BF / RESET_BF)

SET_BF — Set Bit Field

Nome e Objetivo: Seta um campo contínuo de N bits a partir de uma posição inicial (INDEX) em um operando BYTE/WORD/DWORD. Operação atômica em uma instrução.

Quando usar: Inicialização de palavra de comando para múltiplas válvulas, set de máscara de habilitação de eixos, reset parcial de word de status.

Sintaxe e Parâmetros: OUT: operando de destino (BYTE/WORD/DWORD). INDEX: bit inicial (0-based).

LEN: quantidade de bits a setar.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SET_BF AQUI]

```

      +-----+
Init_Valves | SET_BF |
-----| |-----|
"Valve_Cmd".Word--| OUT   |
      0 -----| INDEX  |
      4 -----| LEN    |
      +-----+
```

STL (Statement List):

```

A    "Init_Valves"
L    4
L    0
SET_BF "Valve_Cmd".Word
```

RESET_BF — Reset Bit Field

Nome e Objetivo: Reseta (zera) campo contínuo de N bits a partir de INDEX. Complemento do SET_BF para limpeza seletiva de word de comando.

Quando usar: Parada de grupo de válvulas, limpeza de byte de diagnóstico, desabilitação de faixa de saídas digitais em word consolidada.

Sintaxe e Parâmetros: OUT, INDEX, LEN — idênticos ao SET_BF.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RESET_BF AQUI]

```

      +-----+
Stop_Group  | RESET_BF |
-----| |-----|
"Valve_Cmd".Word--| OUT   |
      0 -----| INDEX  |
      4 -----| LEN    |
      +-----+
```

STL (Statement List):

```

A    "Stop_Group"
L    4
L    0
RESET_BF "Valve_Cmd".Word
```


CAPÍTULO 2 (continuação): Timer Operations

2.5 Instruções Coil de Temporizador (-(TON)-, -(RT)-, -(PT)-)

-(TON)- / -(TOF)- / -(TP)- / -(TONR)- — Coil Timer

Nome e Objetivo: Variantes coil-style dos temporizadores IEC. Integram disparo e instância de timer na mesma rede LAD. O timer referenciado deve ter DB de instância associado via propriedade da coil.

Quando usar: Redes compactas em LAD sem bloco box, código legado S7-300 migrado, sequências visuais em documentação de manutenção.

Sintaxe e Parâmetros: Coil -(TON)-: parâmetros TV (TIME) e bi/timer instance. Saída Q refletida na coil ou em contato posterior. -(PT)-: carrega preset sem iniciar contagem.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO -(TON)- AQUI]

      Start_Delay
-----| |------(TON)
                        TV := T#5s
                        Instance := "DB_TON_Start"

      "DB_TON_Start".Q
-----| |------( ) Motor_Allowed
```

STL (Statement List):

```
// Coil timers são exclusivos LAD; equivalente STL:
CALL "TON", "DB_TON_Start"
    IN := "Start_Delay"
    PT := T#5S
A    "DB_TON_Start".Q
=    "Motor_Allowed"
```

-(RT)- — Reset Timer / -(PT)- — Load Preset

Nome e Objetivo: -(RT)-: zera ET e Q do timer instanciado. -(PT)-: carrega novo PT sem resetar estado de execução — uso avançado em HMI para ajuste de tempo em runtime.

Quando usar: Reset manual de temporizador via IHM, abort de sequência, retargeting de preset em receita sem reiniciar máquina.

Sintaxe e Parâmetros: -(RT)-: associado ao DB de instância do timer. -(PT)-: TV := novo valor TIME.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO -(RT)- AQUI]

      HMI_Reset
-----| |------(RT)
                        Instance := "DB_TON_Start"

      HMI_NewTime
-----| |------(PT)
                        TV := "HMI".Delay_SP
                        Instance := "DB_TON_Start"
```

STL (Statement List):

```
// Reset via CALL com R em TONR ou reinício por IN:=FALSE
A      "HMI_Reset"
R      "DB_TON_Start".IN_Instance_Flag // conforme instância
```

2.6 Controle de Runtime (RE_TRIGR, RUNTIME)

RE_TRIGR — Restart Cycle Monitoring Time

Nome e Objetivo: Reinicia o tempo de ciclo máximo (cycle monitoring timeout) da CPU. Evita erro OB80 por overrun quando bloco de inicialização ou download estende o scan.

Quando usar: Após download parcial, inicialização de arrays grandes, primeiro scan pós-power-on com carga pesada, testes de comissionamento.

Sintaxe e Parâmetros: Sem parâmetros. Chamada incondicional. Executar pontualmente — não deixar em OB1 cíclico.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RE_TRIGR AQUI]

```

      First_Scan
-----| |-----+-----+
                                     |RE_TRIGR |
                                     +-----+
```

STL (Statement List):

```
A      "First_Scan"
CALL   RE_TRIGR
```

RUNTIME — Measure Program Runtime

Nome e Objetivo: Mede tempo de execução em microssegundos de um trecho de código delimitado por duas chamadas RUNTIME consecutivas.

Quando usar: Profiling de OB/FC crítico, diagnóstico de overrun intermitente, validação de otimização de código em S7-1500.

Sintaxe e Parâmetros: MEM (RET_VAL): DINT acumulador. Primeira chamada: inicia medição. Segunda: retorna delta em µs.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RUNTIME AQUI]

```

      +-----+
      | RUNTIME |-- "DB_Diag".T_Start
      +-----+
// ... código medido ...
      +-----+
      | RUNTIME |-- "DB_Diag".T_Delta
      +-----+
```

STL (Statement List):

```
CALL   RUNTIME, "DB_Diag".T_Start
// ... código ...
CALL   RUNTIME, "DB_Diag".T_Delta
```

CAPÍTULO 4 (continuação): Comparator Operations

4.3 Comparadores Variant e Ponteiros (EQ_Type, IS_NULL, IS_ARRAY)

EQ_Type / NE_Type — Comparação de Tipo

Nome e Objetivo: Compara o tipo de dado de um operando VARIANT ou tag genérica com outro tipo referência. Retorna TRUE se tipos coincidem (EQ) ou diferem (NE).

Quando usar: Bibliotecas genéricas de comunicação, FC único para múltiplos tipos de receita, validação de VARIANT recebido via PUT/GET antes de processar.

Sintaxe e Parâmetros: IN1: VARIANT ou tag tipada. IN2: tipo referência (ex. "REAL", struct name). OUT: BOOL.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO EQ_Type AQUI]

```

      +-----+
      | "Rx_Data".Variant | EQ_Type |----- ( ) Type_Real_OK
      | "REAL"          |-----+
      +-----+
  
```

STL (Statement List):

```

CALL  "EQ_Type_Check"
  IN1  := "Rx_Data".Variant
  IN2  := "REAL"
  OUT  := "Type_Real_OK"
  
```

IS_NULL / NOT_NULL — Verificação de Ponteiro

Nome e Objetivo: IS_NULL: TRUE se ponteiro/VARIANT não referencia dado válido. NOT_NULL: inverso. Essencial antes de dereferenciar VARIANT em runtime.

Quando usar: Receitas opcionais, parâmetros de FB não configurados, proteção contra acesso a DB inexistente em endereçamento dinâmico.

Sintaxe e Parâmetros: IN: VARIANT ou Pointer. OUT: BOOL.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO IS_NULL AQUI]

```

      +-----+
      | "Param".Variant | IS_NULL |----- ( ) Param_Missing
      +-----+
  
```

STL (Statement List):

```

CALL  "IS_NULL_Check"
  IN   := "Param".Variant
  OUT  := "Param_Missing"
  
```

IS_ARRAY / EQ_ElemType — Verificação de Array

Nome e Objetivo: IS_ARRAY confirma se operando é array. EQ_ElemType compara tipo dos elementos do array com tipo referência.

Quando usar: Iteração dinâmica com CountOfElements, validação de buffer recebido de SCADA, UDFC genérico para tendências.

Sintaxe e Parâmetros: IN: VARIANT ou array. OUT: BOOL. EQ_ElemType requer IN2 como tipo elemento.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO IS_ARRAY AQUI]

```

      +-----+
      | "Trend".Variant --- | IS_ARRAY |----- ( ) Trend_IsArray
      +-----+
  
```

STL (Statement List):

```

CALL  "IS_ARRAY_Check"
      IN  := "Trend".Variant
      OUT := "Trend_IsArray"
  
```

4.4 Validação de Integridade (|OK| / |NOT_OK|)

|OK| / |NOT_OK| — Check Validity

Nome e Objetivo: Contatos de validação para tags com status de qualidade (S7-1500, tags de comunicação). |OK|: conduz se tag válida. |NOT_OK|: conduz se inválida ou substituída.

Quando usar: Integração com tags OPC/Profinet com quality code, tratamento de valor substituto (substitute value) em falha de sensor, intertravamento em perda de link.

Sintaxe e Parâmetros: Operando: tag com atributo de validade habilitado. Disponível em CPUs com suporte a qualidade de tag.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO OK AQUI]

```

      "AI_Flow".Value
      -----|OK|-----+----- ( ) Flow_Control_En
                        |
      "Comm_Link"
      -----| |-----+
  
```

STL (Statement List):

```

// Verificar validade via bloco de sistema ou qualidade explícita
A    "AI_Flow".Valid
A    "Comm_Link"
=    "Flow_Control_En"
  
```

CAPÍTULO 5 (continuação): Math Functions

5.4 Operações Modulares e Incrementais (MOD, INC, DEC, NEG, ABS)

MOD — Return Remainder of Division

Nome e Objetivo: Retorna resto da divisão inteira $IN1 / IN2$. $ENO = FALSE$ se $IN2 = 0$.

Quando usar: Cálculo de posição em ciclo (índice módulo N), rotação de estação de trabalho, divisão de pacotes em buffer circular.

Sintaxe e Parâmetros: $IN1, IN2$: INT/DINT. OUT: resto. ENO: status.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MOD AQUI]

```

      +-----+
"Station".Idx-----|   MOD   |-- "Station".Pos
      8 -----|           |
      +-----+

```

STL (Statement List):

```

CALL  "MOD_Station"
  IN1 := "Station".Idx
  IN2 := 8
  OUT := "Station".Pos

```

INC / DEC — Increment / Decrement

Nome e Objetivo: Incrementa (INC) ou decrementa (DEC) o valor de IN em 1 e escreve em OUT. Equivalente a ADD/OUT com constante 1, otimizado.

Quando usar: Índice de array, contador manual em HMI, passo de sequência em FC leve.

Sintaxe e Parâmetros: IN/OUT: INT ou DINT. OUT pode ser mesma tag que IN (in-place).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO INC AQUI]

```

      Step_Advance
-----| |-----+-----+
              |   INC   |-- "Seq".StepIdx
"Seq".StepIdx-----|           |
              +-----+

```

STL (Statement List):

```

A    "Step_Advance"
L    "Seq".StepIdx
INC
T    "Seq".StepIdx

```

NEG / ABS — Negation / Absolute Value

Nome e Objetivo: NEG: complemento de dois (negativo). ABS: valor absoluto. ABS de INT_MIN pode overflow — usar DINT intermediário.

Quando usar: Cálculo de erro bidirecional (ABS setpoint - PV), inversão de comando, tratamento de delta em posicionamento.

Sintaxe e Parâmetros: IN: INT/DINT/REAL. OUT: mesmo tipo.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ABS AQUI]

```

      +-----+
      |          |
Error.Value-----|  ABS  |-- Error.Magnitude
      |          |
      +-----+
  
```

STL (Statement List):

```

CALL  "ABS_Error"
      IN   := "Error".Value
      OUT  := "Error".Magnitude
  
```

5.5 Bloco CALCULATE — Expressões Compostas

CALCULATE — Expressão Matemática Configurável

Nome e Objetivo: Bloco com editor de expressão para fórmulas compostas (ex: $(IN1+IN2)*0.5/\sqrt{IN3}$). Reduz cascata de blocos ADD/MUL/DIV.

Quando usar: Engenharia de processo com fórmulas complexas, linearização de curva, cálculo de entalpia ou vazão compensada — documentar expressão no comentário do bloco.

Sintaxe e Parâmetros: EN, ENO. Operandos IN1..INn conforme expressão. OUT: tipo definido na configuração. Erro de domínio (sqrt negativo) → ENO=FALSE.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CALCULATE AQUI]

```

      +-----+
      |          |
Calc_En-----| CALCULATE |
      |          |
-----| |-----|EN      ENO|----- ( ) Calc_Valid
"Flow".Temp-----|          |
"Flow".Press-----|  OUT:   |-- "Flow".Compensated
                  | (IN1+IN2)*0.5
                  |
      +-----+
  
```

STL (Statement List):

```

// CALCULATE é primariamente LAD/FBD; em STL usar CALL com parâmetros
A      "Calc_En"
CALL  "CALC_FlowComp"
      IN1 := "Flow".Temp
      IN2 := "Flow".Press
      OUT := "Flow".Compensated
      ENO => "Calc_Valid"
  
```

CAPÍTULO 6 (continuação): Move Operations

6.3 Operações Ininterruptas e Bit Packing (UMOVE_BLK, SCATTER, GATHER)

UMOVE_BLK / UFILL_BLK — Uninterruptible Move/Fill

Nome e Objetivo: Variantes atômicas de MOVE_BLK e FILL_BLK. Garantem que cópia não seja interrompida por evento assíncrono — consistência de dados em leitura concorrente.

Quando usar: Buffer compartilhado entre OB1 e OB35, cópia de trending para IHM durante scan, dados de segurança em transição.

Sintaxe e Parâmetros: Mesmos parâmetros de MOVE_BLK/FILL_BLK. Maior tempo de bloqueio de CPU — usar apenas quando necessário.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO UMOVE_BLK AQUI]

```

      +-----+
      | Snapshot | UMOVE_BLK |
      +-----+-----+
      |-----| EN      ENO |
      | "Live_Data"-----|
      | "Snap_Data"-----|
      | 100 -----| COUNT |
      +-----+
  
```

STL (Statement List):

```

CALL  "UMOVE_BLK_Snap"
      SRC_ARRAY := "Live_Data"
      DST_ARRAY := "Snap_Data"
      COUNT    := 100
  
```

SCATTER / GATHER — Bit Sequence Pack/Unpack

Nome e Objetivo: SCATTER: decompõe sequência de bits de IN em array de BOOL. GATHER: compacta array de BOOL em word/dword de saída.

Quando usar: Interface com word Modbus de status, conversão de 16 alarmes em word para HMI, packing de DI em DWORD para transmissão.

Sintaxe e Parâmetros: SCATTER: IN (bit sequence), OUT (array of BOOL), COUNT. GATHER: inverso.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SCATTER AQUI]

```

      +-----+
      | "Alarm".Word-----| SCATTER |
      | "Alarm".Bits[ ]---|
      | 16 -----| COUNT |
      +-----+
  
```

STL (Statement List):

```

CALL  "SCATTER_Alarms"
      IN    := "Alarm".Word
      OUT   := "Alarm".Bits
      COUNT := 16
  
```

6.4 Variant e Limites de Array (VariantGet, LOWER_BOUND, UPPER_BOUND)

VariantGet / VariantPut

Nome e Objetivo: VariantGet: lê valor apontado por VARIANT para tag tipada. VariantPut: escreve valor em destino VARIANT.

Quando usar: FC genérico de cópia, logging polimórfico, receita com campos de tipos variados.

Sintaxe e Parâmetros: VARIANT: ponteiro genérico. VALUE: tag com tipo concreto. ENO=FALSE se tipo incompatível.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO VariantGet AQUI]

```

+-----+
"Src".Variant-----| VariantGet |
"Dst".Real-----|             |
+-----+
```

STL (Statement List):

```
CALL "VariantGet_Real"
  VARIANT := "Src".Variant
  VALUE   := "Dst".Real
```

LOWER_BOUND / UPPER_BOUND / CountOfElements

Nome e Objetivo: Retornam índice inferior, superior e quantidade de elementos de ARRAY[*] dinâmico ou VARIANT array.

Quando usar: Loop FOR com limite dinâmico, validação de bounds antes de acesso, UDFC para processamento de buffer de tamanho variável.

Sintaxe e Parâmetros: ARR: ARRAY[*] ou VARIANT. OUT: DINT. ENO=FALSE se não for array.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO LOWER_BOUND AQUI]

```

+-----+
"Buf".Array[]-----| LOWER_BOUND |-- "Loop".Idx_Min
+-----+

+-----+
"Buf".Array[]-----| UPPER_BOUND |-- "Loop".Idx_Max
+-----+
```

STL (Statement List):

```
CALL "LOWER_BOUND_Buf"
  ARR := "Buf".Array
  OUT := "Loop".Idx_Min

CALL "UPPER_BOUND_Buf"
  ARR := "Buf".Array
  OUT := "Loop".Idx_Max
```


CAPÍTULO 8 (continuação): Program Control

8.3 Lista de Saltos (JMP_LIST) e Diagnóstico Local (GET_ERROR)

JMP_LIST — Define Jump List

Nome e Objetivo: Define lista de destinos LABEL para distribuição de salto por índice K. Alternativa estruturada ao SWTCH em projetos grandes.

Quando usar: Sequenciador com >8 ramos, tabela de saltos documentada, máquina de estados tabular.

Sintaxe e Parâmetros: K: INT índice. Lista de pares (valor, LABEL).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO JMP_LIST AQUI]

```

+-----+
"State".Idx-----| JMP_LIST |
                   | K       |
                   | 1: St_10 |
                   | 2: St_20 |
                   | 3: St_30 |
+-----+
```

STL (Statement List):

```

L      "State".Idx
JMP_LIST
1  St_10
2  St_20
3  St_30
```

GET_ERROR / GET_ERR_ID — Local Error Capture

Nome e Objetivo: Captura erro local (último ENO=FALSE em bloco) sem depender de OB80/OB82. GET_ERR_ID retorna código numérico do erro.

Quando usar: Diagnóstico em FC de comunicação, log de falha de CONVERT/MOVE sem parar CPU, IHM de manutenção com último erro.

Sintaxe e Parâmetros: REQ: BOOL trigger. ERR_STRUCT: UDT de erro. DONE/ERROR: status.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO GET_ERROR AQUI]

```

      Diag_Req
-----| |-----+-----+
                   | GET_ERROR |
                   |           |
"Diag".ErrStruct--| ERR       |
                   +-----+
```

STL (Statement List):

```

A      "Diag_Req"
CALL   "GET_ERROR"
      REQ      := TRUE
      ERR_STRUCT := "Diag".ErrStruct
```

8.4 Controle de Execução (STP, ENDIS_PW)

STP — Exit Program

Nome e Objetivo: Transiciona CPU para STOP mode a partir do programa. Irreversível sem intervenção externa (online, cartão, PG).

Quando usar: Condição de catástrofe com dano mecânico iminente, teste de falha controlado em bancada — nunca em produção sem análise de risco.

Sintaxe e Parâmetros: Chamada incondicional ou condicionada. Sem parâmetros.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO STP AQUI]

```

Catastrophic
-----| |-----+-----+
                                   | STP |
                                   +-----+
```

STL (Statement List):

```

A    "Catastrophic"
CALL STP
```

ENDIS_PW — Limit and Enable Password Legitimation

Nome e Objetivo: Habilita/desabilita proteção por senha de níveis de acesso. LIMIT restringe tentativas; ENABLE ativa legitimação.

Quando usar: Comissionamento: desabilitar proteção temporariamente; produção: reabilitar. Documentar em procedimento BLBW de entrega.

Sintaxe e Parâmetros: REQ, PASSWORD (conforme versão). Consultar manual de proteção CPU.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ENDIS_PW AQUI]

```

Comm_Enable
-----| |-----+-----+
                                   | ENDIS_PW |
                                   +-----+
```

STL (Statement List):

```

// Procedimento sensível – executar apenas em startup controlado
CALL "ENDIS_PW_Enable"
    REQ := "Comm_Enable"
```

APÊNDICE A: Arquitetura de Execução e Ciclo de Scan

A.1 Hierarquia de OBs e Prioridade de Saídas

Modelo de Prioridade OB — Referência BLBW

Nome e Objetivo: OB1 (cyclic) executa após eventos de maior prioridade. OB35/OB30 (cyclic interrupts) preemptam OB1. OBs de erro (OB80-OB87) e startup (OB100) têm prioridade definida por classe.

Quando usar: Dimensionamento de scan time, decisão de alocar PID em OB30 vs OB1, diagnóstico de jitter em contagem de alta velocidade.

Sintaxe e Parâmetros: N/A — conceito arquitetural. Saídas escritas em OB de maior prioridade sobrescrevem OB1 no mesmo ciclo até próxima atualização.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
// Estrutura recomendada BLBW:
// OB100  - Startup, inicialização DB, RE_TRIGR
// OB35   - PID, contagem rápida, filtros
// OB1    - Sequência, lógica, comunicação
// OB82   - Diagnóstico de módulo (template BLBW_Diag)
```

STL (Statement List):

```
// OB35 - exemplo de prioridade sobre OB1
ORGANIZATION_BLOCK OB35
BEGIN
    CALL  "FC_PID_Fast"
    CALL  "FC_Counter_HSC"
END_ORGANIZATION_BLOCK
```

A.2 Process Image vs Acesso Direto

%I / %Q / PEEK / POKE — Acesso a Periféricos

Nome e Objetivo: Process Image: entrada lida no início do ciclo, saída escrita no fim — consistente mas com 1 ciclo de latência. Acesso directo (%I, %Q ou PEEK/POKE): imediato, risco de inconsistência intra-scan.

Quando usar: PEEK/POKE: diagnóstico, leitura pontual fora do image. %I/%Q: otimização S7-1500. Não misturar image e direct no mesmo bit sem documentar.

Sintaxe e Parâmetros: %I0.0, %Q0.0 — sintaxe absoluta. PEEK/POKE: área, DB, offset, bit.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
// Leitura otimizada S7-1500
%I10.0
-----| |----- ( ) %Q10.0
```

STL (Statement List):

```
A    %I10.0
=    %Q10.0

// PEEK - leitura imediata de entrada
L    PEEK_BOOL
    AREA := 16#82    // Inputs
    DBNO := 0
```

```
BYTE := 10  
BIT  := 0  
T    "Diag".In_Immediate
```

APÊNDICE B: Referência Rápida LAD ↔ STL

B.1 Tabela de Equivalência de Instruções

Equivalência LAD ↔ STL — Principais Instruções

Nome e Objetivo: Tabela de referência cruzada para conversão manual entre LAD e STL em manutenção de campo sem TIA Portal disponível.

Quando usar: Análise de print de programa em papel, suporte remoto, auditoria de código legado S7-300.

Sintaxe e Parâmetros: Ver tabela abaixo nos exemplos LAD/STL.

Exemplo Prático de Aplicação:

LAD (Ladder):

LAD Elemento	STL	Observação
(NA)	A	AND com RLO anterior
(NF)	AN	AND NOT
NOT	NOT	Inverte RLO
()	=	Atribuição
(S)	S	Set
(R)	R	Reset
P	FP Mem	Borda subida
N	FN Mem	Borda descida
CMP >=	>=I >=D >=R	Sufixo por tipo
TON box	CALL TON	DB instância obrigatório

STL (Statement List):

```
// Exemplo de conversão rede LAD → STL
// LAD: |Emerg|---+---( ) Run
//      |Start|---+
A      "Emerg"
A      "Start"
=      "Run"
```

B.2 Convenções de Nomenclatura BLBW

Padrão de Tags BLBW — SanBox / Industrial

Nome e Objetivo: Convenção corporativa BLBW para nomenclatura de tags, garantindo legibilidade entre projetos SanBox, painéis e estações de bombeamento.

Quando usar: Todo novo projeto BLBW. Migração de código legado deve mapear tags antigas para padrão na primeira intervenção.

Sintaxe e Parâmetros: Estrutura: [Área][Equipamento][Função][Modificador]. Ex: PU_01_Run_Cmd, TK_02_Level_PV, AL_Flow_HiHi.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
// Exemplos de tags BLBW em rede LAD:
      PU_01_Run_FB
-----| |-----+----- ( ) PU_01_Run_Cmd
      |
      PU_01_Fault
-----|/|-----+
```

```
TK_02_Level_LL
-----| |----- (S) AL_TK02_Level_LL
```

STL (Statement List):

```
// Prefixos BLBW padrão:
// PU = Bomba (Pump)      TK = Tanque (Tank)
// XV = Válvula ON/OFF   CV = Controle (válvula modulante)
// AL = Alarme            AI/AO = Analógico
// FB = Feedback campo   Cmd = Comando

A      "PU_01_Run_FB"
AN     "PU_01_Fault"
=      "PU_01_Run_Cmd"
```